TEKNOLOGI IOT II

Disusun oleh :

Nama: Kezia Dias Ateta Br Ginting

NPM :25782042

Kelas: TRI-3B

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra,

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

3.1 Tujuan Pembelajaran

- Memahami perbedaan mode jaringan pada ESP8266 (*Station, Access Point*, dan *Dual Mode*).
- Menggunakan NodeMCU ESP8266 sebagai Web Server untuk menyajikan antarmuka halaman HTML tertanam (*embedded HTML*).
- Menggunakan NodeMCU ESP8266 sebagai HTTP Client untuk mengirimkan instruksi ke *server* lain.
- Memahami implementasi arsitektur *Client-Server* menggunakan metode interaksi HTTP GET.

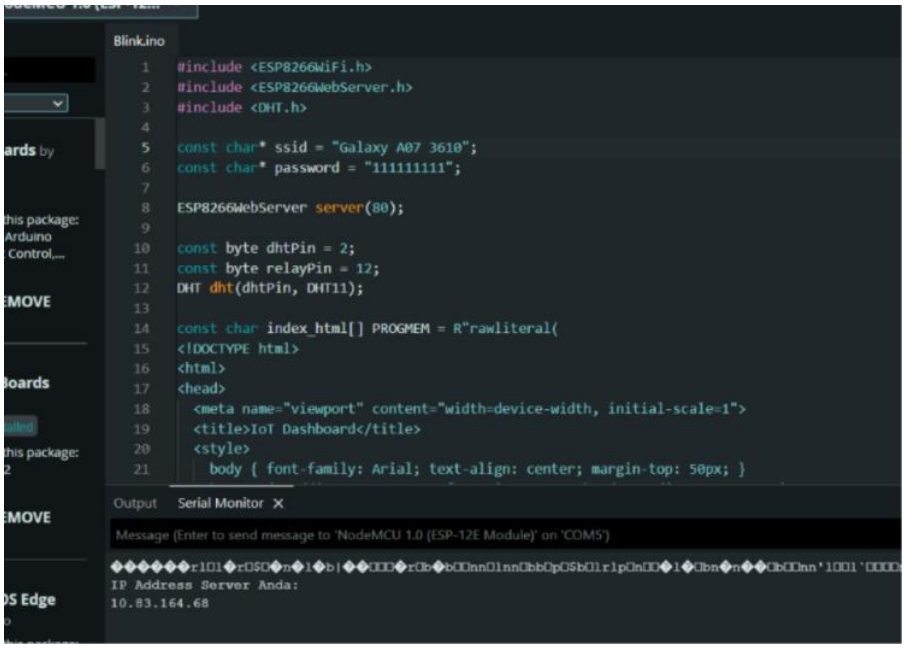
3.2 Teori Dasar

- Mode Jaringan ESP8266: Cip ESP8266 mendukung tiga mode konfigurasi Wi-Fi, yaitu Station (STA) di mana NodeMCU bertindak sebagai klien yang terhubung ke *Router/Access Point* luar, Access Point (AP) di mana ia memancarkan SSID sendiri, serta Dual Mode (STA + AP) yang menggabungkan keduanya.
- Arsitektur Client-Server & HTTP: Komunikasi web dikendalikan oleh protokol HTTP melalui metode *Request* (permintaan) dan *Response* (tanggapan). *Server* bertindak pasif mendengarkan di port 80, sedangkan *Client* secara proaktif menghubungi server. Metode HTTP GET digunakan untuk meminta data atau memicu aksi perangkat keras melalui *endpoint* URL khusus.

3.3 Langkah Kerja Praktikum

Praktikum 1: NodeMCU sebagai Web Server (Menyajikan HTML)

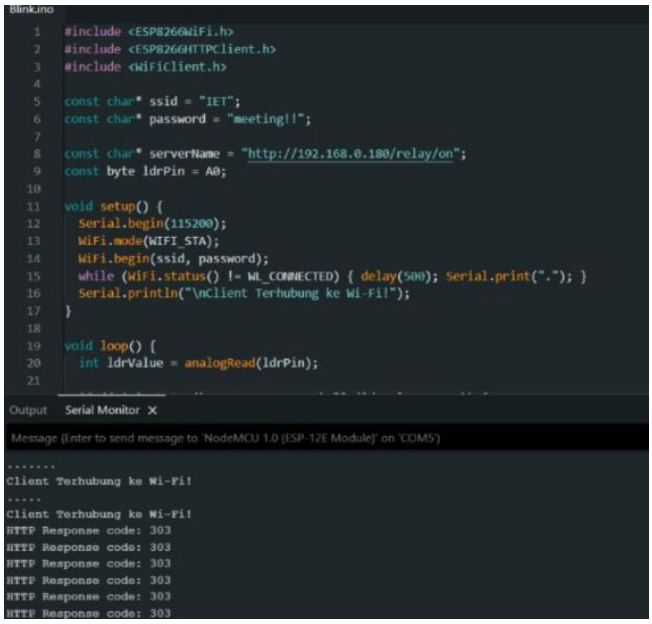
- Rangkaian Perangkat Keras:
    - Pin DATA sensor DHT dihubungkan ke pin D4 (GPIO 2).
    - Pin Signal Relay dihubungkan ke pin D6 (GPIO 12).
- Penjelasan Program: Menggunakan pustaka ESP8266WebServer untuk menjalankan server di port 80. Teknik *Raw String Literal* C++ (R"rawliteral(...)rawliteral") digunakan untuk menulis kode HTML multibaris dengan mudah. Fungsi handleRoot() bertugas membaca suhu dari sensor DHT, mengganti teks *placeholder* %TEMPERATURE% secara dinamis, lalu mengirimkannya ke peramban (*browser*).

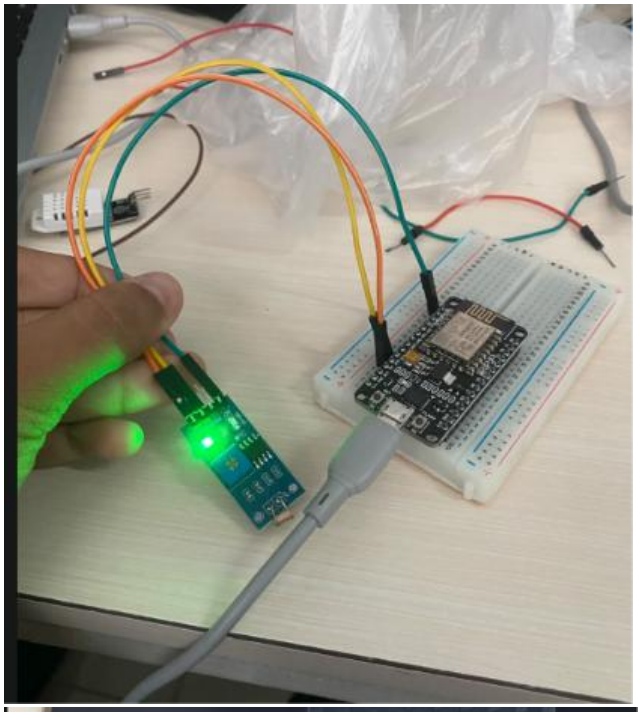


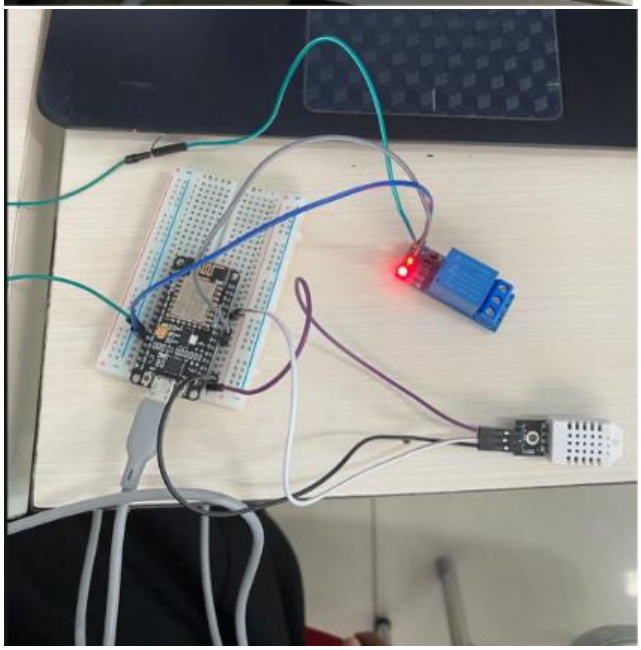


Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)

- **Rangkaian Perangkat Keras:** Sensor LDR dan Resistor 10k Ohm dirangkai sebagai pembagi tegangan dengan output jalur sinyal ke pin **A0**.
- **Penjelasan Program:** Perangkat dikonfigurasi sebagai klien yang memantau nilai sensor LDR. Ketika nilai pencahayaan mendeteksi kegelapan (ldrValue < 300), klien secara otomatis mengirimkan permintaan HTTP GET (http.GET()) jarak jauh ke alamat IP *Server* (NodeMCU Praktikum 1) untuk memicu aktivasi *Relay*.







3.4 Jawaban Latihan

- **Latihan 1 (Pemahaman Jaringan):** Mengapa kita menggunakan mode jaringan *Station (STA)* dan bukan *Access Point (AP)* pada praktikum ini? *Jawab:* Karena kedua perangkat NodeMCU (server dan client) perlu berada dalam satu jaringan lokal (*subnet*) yang sama serta memerlukan jalur komunikasi yang terhubung ke satu *Access Point* utama (seperti *Hotspot* HP) agar dapat saling bertukar paket data IP secara langsung. Jika menggunakan mode AP, perangkat akan terisolasi dan tidak bisa saling mengakses jaringan satu sama lain tanpa *router* perantara.

- **Latihan 2 (Modifikasi Antarmuka & Telemetri):** Potongan kode HTML dan fungsi handleRoot() untuk kelembapan dan tombol toggle tunggal:

```
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>IoT Dashboard</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 50px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor:
pointer;}
    .btn-on { background-color: #4CAF50; color: white; border: none; }
    .btn-off { background-color: #f44336; color: white; border: none; }
    .sensor-box { font-size: 24px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>ESP8266 Web Server</h1>
  <div class="sensor-box">
    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
    <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
  </div>
  <h2>Kendali Relay</h2>
  %RELAY_BUTTON%
</body>
</html>
)rawliteral";

void handleRoot() {
  String html = index_html;
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  html.replace("%TEMPERATURE%", isnan(t) ? "--" : String(t));
  html.replace("%HUMIDITY%", isnan(h) ? "--" : String(h));

  int relayState = digitalRead(relayPin);
  String buttonHtml = "";
  if (relayState == LOW) {
    buttonHtml = "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN
(ON)</button></a>";
  } else {
    buttonHtml = "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN
(OFF)</button></a>";
  }
```

```
  html.replace("%RELAY_BUTTON%", buttonHtml);

  server.send(200, "text/html", html);
}
```

- **Latihan 3 (Analisis Kode Client):**

Apa yang akan dicetak oleh *Serial Monitor* jika NodeMCU *Server* mendadak kehilangan daya (*offline*)?

Jawab: Fungsi http.GET() akan gagal terhubung dan mengembalikan nilai kode respons negatif (seperti nilai error koneksi atau kode kesalahan komunikasi jaringan dari pustaka HTTPClient).

- **Latihan 4 (Pengembangan Algoritma):** Modifikasi blok if pada Klien agar mengirim permintaan HTTP GET **hanya jika** suhu melebihi 35°C:

```
float t = dht.readTemperature();
if ((WiFi.status() == WL_CONNECTED) && (!isnan(t)) && (t > 35.0)) {
  WiFiClient client;
  HTTPClient http;
  http.begin(client, serverName);
  int httpResponseCode = http.GET();
  http.end();
  delay(10000);
}
```

3.5 Tugas: Pengembangan Web Server Cerdas dan Analisis Arsitektur

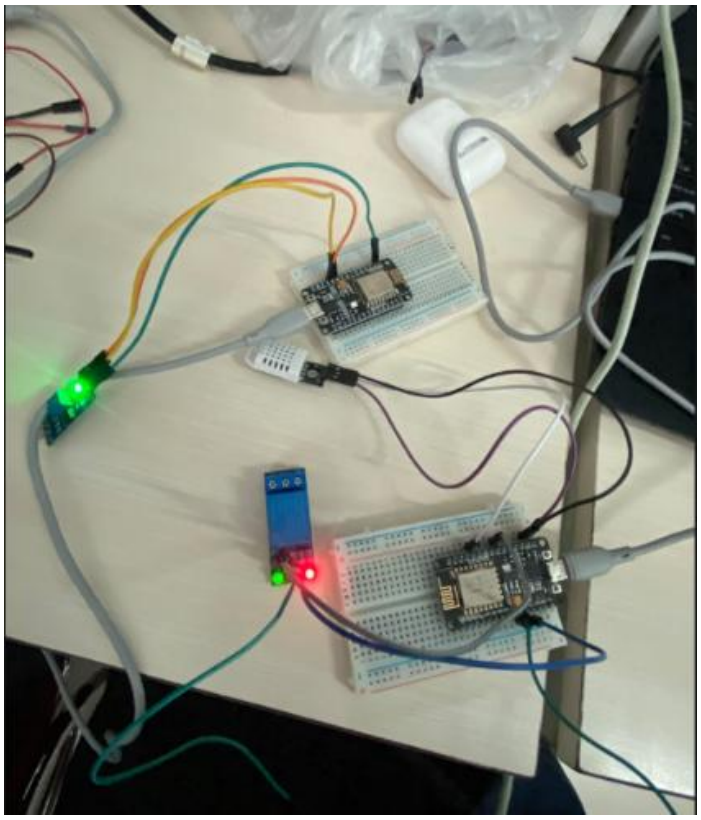
- **Implementasi Kode Server dengan Auto-Refresh:** Untuk mencapai interaktivitas tampilan otomatis, disisipkan tag meta HTML di dalam bagian <head> pada variabel index_html:

C++

```
<meta http-equiv="refresh" content="5">
```

Tag ini memaksa *browser* melakukan pemuatan ulang halaman secara otomatis setiap 5 detik.

- **Analisis Arsitektur dan Evaluasi Kelemahan *Meta Refresh*:** Meskipun nilai suhu di peramban dapat berganti otomatis tanpa mengeklik tombol *refresh* secara manual, metode *Meta Refresh* HTTP konvensional memiliki beberapa kelemahan signifikan:
    1. **Gejala Layar Berkedip (*Flickering*):** Setiap interval 5 detik habis, halaman web mengalami *full-page reload* yang menyebabkan layar berkedip putih secara berkala dan menurunkan kenyamanan visual pengguna.
    2. **Pemborosan *Bandwidth*:** Peramban harus mengunduh ulang seluruh kerangka HTML, gaya CSS, dan struktur teks dari awal secara terus-menerus meskipun tidak ada perubahan nilai data yang berarti.
    3. **Potensi Bentrokan Waktu (*Race Condition*):** Jika pengguna menekan tombol *Relay* (ON/OFF) bertepatan dengan detik ke-5 saat proses *auto-refresh* berlangsung, perintah klik dapat terganggu atau mengalami kegagalan kirim karena server sedang memproses ulang koneksi.



NodeMCU 1.0 (ESP-12...

Blink.ino

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

const char* ssid = "IE!";
const char* password = "meeting!!";

const char* serverName = "http://192.168.0.180/relay/on";
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nClient Terhubung ke Wi-Fi!");
}

void loop() {
  int ldrValue = analogRead(ldrPin);
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

```
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: -5
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
```

3.6 Kesimpulan

Praktikum Modul 3 ini berhasil membuktikan bahwa NodeMCU ESP8266 dapat dioperasikan secara optimal sebagai *Web Server* mandiri berbasis antarmuka HTML dinamis maupun sebagai *HTTP Client* yang melakukan interaksi jarak jauh melalui protokol HTTP GET. Meskipun arsitektur HTTP tradisional memiliki keterbatasan performa *real-time* (seperti isu *flickering* pada penyegaran berkala), protokol ini tetap menjadi fondasi universal yang sangat handal untuk membangun sistem interkoneksi perangkat cerdas berbasis IoT.